# A Crash-Proof Team of Coding Agents

*Claude Code remembers the conversation; Temporal remembers the job. Compose the two and you get a durable delivery team: nine single-purpose jobs carrying an issue from filed to merged, with Claude Code doing the judgment work inside them — building, reviewing, resolving — under skill playbooks, enforced hooks, and human-controlled settings on every retry. We SIGKILLed a worker mid-task and the run finished as the same session for four cents; the team resolved a real merge conflict on the way, with no human decision in the loop.*

---


A long headless run of an AI coding agent (headless: one command in a terminal, no chat window) gets interrupted routinely: a worker restarts, the API returns an overload error, a stream drops. There is a moment in every infrastructure demo where you stop trusting the slides and ask the presenter to pull the plug, so we opened this project by pulling it ourselves.

We killed the worker's whole process tree three minutes into a task, with no completed result saved anywhere. A different process on a restarted worker picked up the same half-finished conversation and ran it to completion as the same session.

That is the durability half of this article, and it is the smaller half. The bigger half is what durability buys: a durable team of nine single-purpose jobs, split into lanes with a real division of labor and one clear mandate apiece. One of the nine runs Claude Code itself, and it does all the judgment work: building the feature, reviewing the diff, resolving the conflict. The other eight carry the delivery pipeline around it, from opening the pull request to pushing a resolved branch. Each lane's agent is equipped with its team's skills: actual playbook files it discovers and invokes, not adjectives in a prompt. It is governed by hooks and deny rules the workspace re-asserts before every chunk (one bounded slice of a run, capped at a fixed number of agent turns): an org-wide floor identical for every lane, every retry, every resume, plus the watched rules and finish gate each team commits for itself. That team carried a filed GitHub issue all the way to a merged pull request, resolving a real merge conflict along the way, with no human decision in the loop.

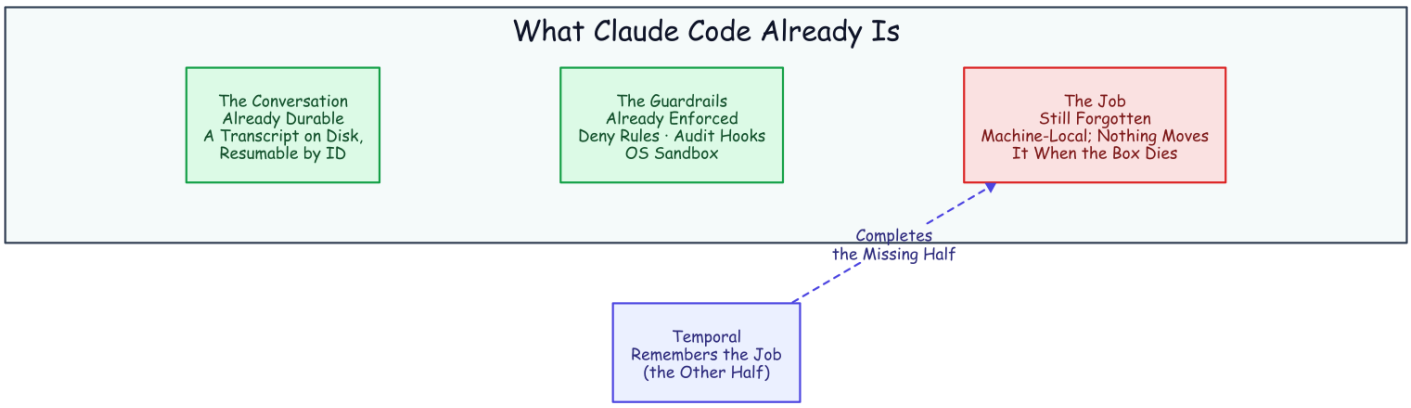
*The fine print, up front: one engineer ran everything here, and the "we" throughout is editorial. The agent model in every measured run is `haiku`, Claude's cheap fast tier, in July 2026; the tasks are benchmark-sized and the run counts are single-digit, so every number is a point estimate, not a distribution. Every number also traces to an evidence file in the public repository, thumarrushik/temporal-claude-demo, with the runners under `deploy/`. The dollar figures are haiku-priced, too: on a stronger model tier, every absolute number here scales up roughly an order of magnitude.*

## Claude Code Is Already Half a Durable System

Run Claude Code headless and it executes the whole agent (reading files, editing them, running the tests), prints a result, and exits. That result carries a **session ID**. Hand the ID back later with a resume flag and the agent reopens that exact conversation with everything it had learned, because the transcript is an ordinary file on disk, filed under the directory the agent ran in. The *conversation* is already durable. And this is not a toy mode: the official GitHub Action ships this same headless agent into continuous integration, where it reviews pull requests and fixes failing checks, an Action built on the very agent framework this project uses.[1][2]

It also ships a real guardrail harness: the part that matters the moment you hand an agent a shell. The dangerous commands are denied by the tool itself, not merely discouraged in a prompt: deleting a tree, escalating privileges, pushing to a remote. A hook fires after every single tool call and can veto it and log it. The operating system sandboxes what the process can touch. Skills and a required output schema shape what the agent does and what it must return.[3] A line in a prompt is a suggestion. A deny rule is a fact, identical on the first attempt and the sixth.

That is one half of a durable system, and it is the hard, conversational half. The other half its own documentation quietly concedes. A resumed session is local to the machine and the directory it was born in. There is no way to move it to another box, no lease, nothing that notices the machine died and carries the work elsewhere.[4] The agent remembers the conversation. **Nothing remembers the job.**




*Half a Durable System. Two of the three pieces already exist. The job is the piece Claude Code leaves on the floor: the thing that would notice a dead machine and move the work.*

## The Job Is the Half That Dies

The obvious way to drive a headless agent is a loop. Ask it to continue, refresh the session ID from the result, ask it to continue again:

```
while :; do claude -p "continue" --resume "$SID" --max-turns 6; done
```

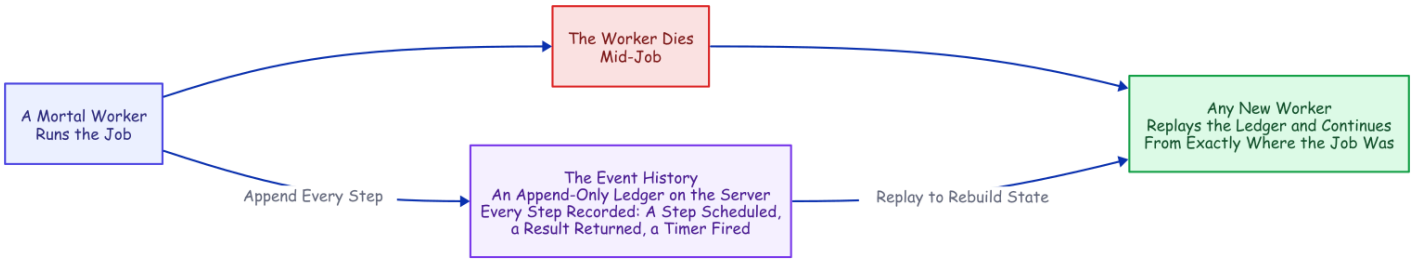
The real thing parses each run's output to keep the session ID current, but that is the shape. It works right up until the process holding it dies. And when that process dies, exactly one thing survives, because it is a file on disk: the transcript. Everything else was memory. The current session ID. The retry count. The instruction someone typed an hour ago. The bare fact that a job was running at all.

An agent run is precisely the kind of job you cannot afford to forget. It runs for hours. It bills by the minute. It carries context it took real work to accumulate. It touches real systems. And it tends to fail while unattended. Remember it yourself and you end up hand-building durable state, a single-writer lock, a retry taxonomy, a liveness probe, and a status endpoint, until you own a distributed system you never meant to design. Or you can give the job the one thing the conversation already has: a record that outlives the process.

## The Missing Half Has a Name

Strip the agent out of that hand-built list and what is left over (durable state, one writer at a time, retries, timeouts, liveness, a way to see what happened) is not an AI problem at all. It is the standard checklist for any long job that has to outlive its own hardware, and the industry has a name for the answer: **durable execution**. Temporal, the engine we used, grew out of a system built at Uber to run exactly these long, crash-prone, many-step jobs.[5]

The core move is a refusal to trust process memory. A database does not keep your data safe by keeping its process alive; it writes every change to a log and rebuilds after a crash by replaying it. Durable execution does the same thing for *program control flow*. Every step a job takes (a step scheduled, a result returned, a timer fired, a message received) is appended to a ledger on the server, the **event history**. Kill the process and a new one reads that history back and continues from exactly where the job was.




*Durable Execution. The worker is disposable on purpose. It appends every step to the history as it goes; when it dies, any other worker replays the ledger and picks up mid-job. No memory snapshot required.*

The trick that makes the reconstruction work is **deterministic replay**. Temporal never photographs memory. It re-runs the job's code from the first line, and at every step that already happened, it substitutes the recorded result instead of doing the work again. Same code, same recorded inputs, same decisions: the replay lands in the same state. The catch lives in the word *same*. One clock reading, one coin flip, one network call in the replayed layer, and the replay diverges from its own history and the whole trick breaks.

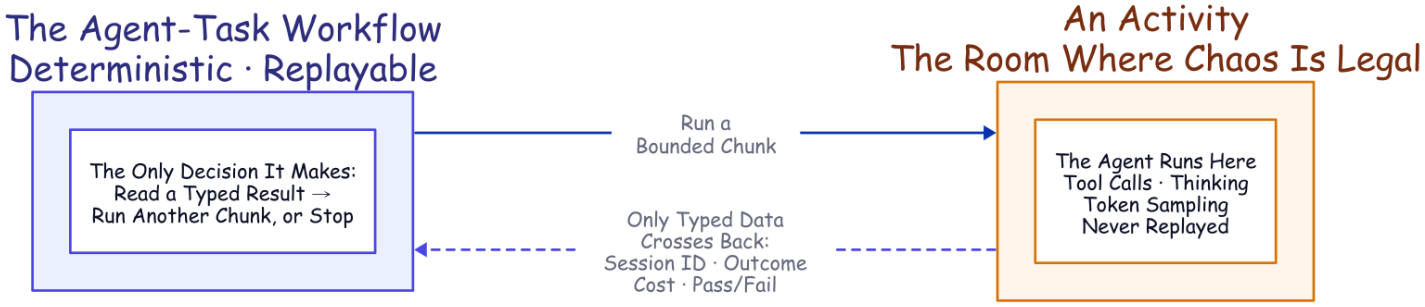
That constraint forces a clean split, and three plain words carry the rest of this article. A **workflow** is the code that decides what a job does next: the deterministic, replayable layer, the part that must never flip a coin. A **worker** is an ordinary process on some machine that physically does the work, and it is meant to be mortal. An **activity** is a single step that the engine treats as a sealed box: it may call the network, write files, roll dice, anything, because Temporal never replays an activity's insides. Each step retries under a policy you declare rather than code. And while a step runs, it can send **heartbeats**: small liveness pulses that may carry a little data with them, so that if the pulses stop for too long, the server declares that attempt dead and starts a retry that can read the last pulse the dead attempt sent.[6]

That last capability, a retry that can read the dead attempt's final heartbeat, is the mechanism the rest of this design rests on.

## Put the Agent Where Non-Determinism Is Legal

One problem looks fatal at first. A workflow has to be deterministic, and an AI coding agent is the least deterministic software you will ever run: same prompt, different transcript, every time. Put the agent inside workflow code and the first replay diverges on the first token. That is a category error, not a knob you can tune.

But durable execution already has a room where non-determinism is not just tolerated but expected: the activity. Seal the entire agent inside an activity and the workflow never sees the chaos. All it sees is what crosses back as plain data: a session ID, an outcome, a dollar cost, a validated pass/fail report. Its own logic collapses to something a database could run: read a typed result, decide *continue or stop*, repeat. The tool calls and the token sampling never touch the replayable layer.




*The Seam. The workflow stays replayable because the agent is sealed inside an activity. Only typed data crosses the line, so the workflow's own decisions never depend on anything the agent did unpredictably.*

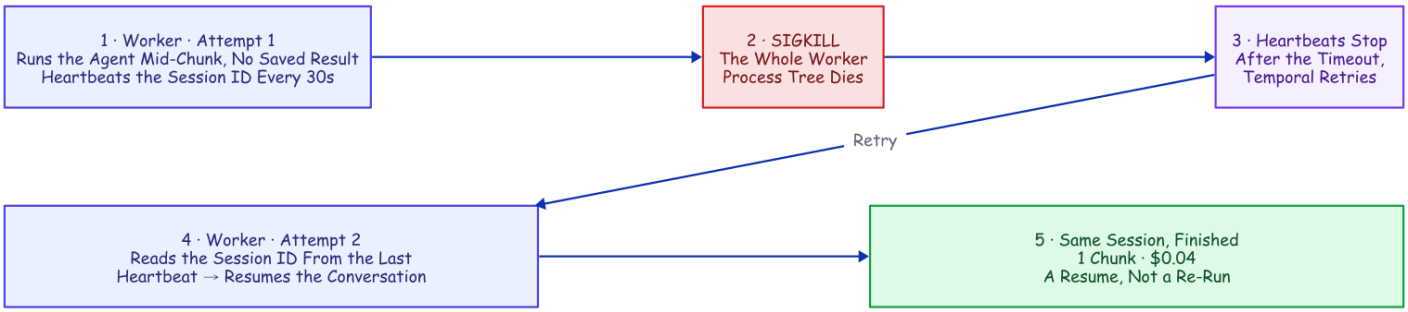
Two details make resume survive that boundary. First, each run gets one stable working directory, derived from the job's own identity, so every attempt lands in the same folder, and because the agent files its transcript under the directory it ran in, landing in the same folder is exactly what lets a retry find the same conversation again. Second, the workflow always chains forward the latest session ID an activity hands back, because a resume can mint a fresh one. What the job's ledger actually stores is a *pointer* into the conversation's transcript. That pointer is very nearly the entire integration.

## Recovering a Crashed Run from Its Last Heartbeat

This is where the sealed box earns its place.

While the agent works, a timer inside the activity fires a heartbeat every thirty seconds no matter what the agent is doing. Deliberately dumb: a pulse tied to the agent's own output would make a long, silent tool call (a slow test suite, a dependency install) look identical to a dead worker. A timer removes the ambiguity. If the heartbeats stop, the worker's event loop actually stopped, and the two-minute heartbeat timeout trips. The opposite failure, an agent still alive and still pulsing but wedged in a loop or a hung tool, never trips that timeout, so a separate, longer ceiling on the whole chunk bounds that case instead: two timeouts for two distinct ways to be stuck. Every pulse carries the latest known session ID, which the agent announces at the very start of a run.

Now kill the worker. The pulses stop. After the heartbeat timeout the server declares the attempt dead and schedules a retry, and the retry reads the session ID straight out of the dead attempt's last heartbeat and relaunches the agent with a resume. No completed checkpoint is required; the last heartbeat serves as the checkpoint.




*Recovering a Crashed Run from Its Last Heartbeat. Attempt 2 recovers the session ID from the dead attempt's final pulse, no completed result required, and resumes the same conversation. A re-read, not a re-run.*

We proved it the blunt way. We started a run, waited until the agent was mid-task and actively writing files (the session live, its ID already heartbeated), and then killed the worker's **entire process tree** with an unblockable signal, leaving no completed result to fall back on. A restarted worker was already polling. After

the timeout, Temporal retried the step, and the evidence of what happened is a single line the recovered run left in its workspace:[7]

```
{"event": "resume_session_from_heartbeat", "attempt": 2,
 "input_session_id": null, "heartbeat_session_id": "698c432a-…"}
```

The input session ID is null: the workflow had no completed chunk to hand back. The ID came *only* from the heartbeat. The run finished as the **same** session in one chunk. Its total cost was $0.0404, and that figure is not a recovery penalty; it is the ordinary cost of the resumed session re-reading its own accumulated context at the cache rate and then finishing the work. (One accounting honesty: whatever the dead attempt burned before the kill never returned a result, so it appears in no ledger; the figure is the surviving attempt's bill.) The durability machinery itself added no tokens.

Three things have to be right, and each is a caveat worth stealing. **The heartbeat has to be recorded before the crash.** The SDK worker throttles how often heartbeat details are actually recorded to the server, and the default throttle is too coarse to catch a short chunk, so this project's worker tightens it on purpose.[8] **The worker has to die as a whole group.** Kill only the parent and the agent's child process is orphaned, finishes the work anyway, and *masks* the recovery: our first recovery demos "succeeded" exactly this way, and they were lying, more than once, before we caught it. **And the crash has to land while the chunk is genuinely in flight.** Get those right and a dead worker becomes one line in the history that the workflow code never even mentions.

The everyday value is not that deliberate kill; a worker rarely dies outright. What actually stops a headless run is the API on the other end: an overload during a busy hour, a rate limit, a dropped stream. The agent surfaces those on its result; the activity raises them as typed, retryable failures; the retry policy backs off (five seconds, doubling to a two-minute cap, six attempts) and every retry *resumes* the conversation instead of restarting the task. An overload window becomes added latency rather than a failed run.

Point that same machinery at a whole repository and it does something bigger. The heartbeat, the resume, and the declared retries that just carried one agent through a crash are what will carry a whole *team* of agents from a filed issue to a merged pull request, resolving a real merge conflict along the way. That team is the back half of this article. But every agent on it runs inside the same sealed box you just watched recover, so it is worth opening that box to see exactly how one is built.

## How a Chunk Actually Runs

Open the box and one chunk is a single ordinary function. The workflow calls it, the function runs the agent, and what comes back is a typed record: a session ID, an outcome, a cost, a turn count, the validated report, and nothing else. That typing is the wall. The workflow can only read fields on that record, so nothing the agent did unpredictably has a path into the replayable layer.

Launching the agent inside that function is a short list of settings, and one of them does the quiet heavy lifting: the agent is told to load its configuration from the project directory *only*, never from the machine's own user config. That single choice is why the policy below is true rather than hopeful: the rules ride inside the workspace, so the same agent runs on every worker on every machine. The rest read as you would guess: resume the prior session, cap the run at a fixed number of turns so a chunk stops cleanly with its transcript intact, auto-accept file edits but gate every other tool behind an explicit allow-list, and force the closing report to match a schema so the workflow reads a real pass/fail value instead of parsing prose.

The policy itself is a small settings file (`settings.json`). Humans control it: the rules live in version-controlled harness code, and the workspace re-stamps them fresh at the start of every chunk:

```
{
  "permissions": {
    "deny": ["Bash(rm -rf:*)", "Bash(sudo:*)", "Bash(git push:*)"]
  },
  "hooks": {
    "PostToolUse": [
      { "matcher": "*", "hooks": [{ "type": "command", "command": "cat >> .claude/hook-
log.jsonl" }] },
      { "matcher": "*", "hooks": [{ "type": "command", "command": "python3 .claude/flag-
rules.py" }] }
    ]
  }
}
```

Read the deny list as architecture, not only safety. Deleting a tree and escalating privileges are the obvious entries. `git push` is the interesting one: the agent writes code and is never allowed to reach a remote, so every push and every merge in this entire system is done by the harness, in its own step, with its own credentials. The first hook appends every tool call to an audit log as it happens; the second flags a wasteful pattern a later experiment taught us to catch. And because the workspace rewrites this file before *every* chunk, the deny rules and the hooks are byte-for-byte the same on every attempt, through every retry and every resume. The guardrail cannot drift.

The settings file is only the smallest piece of what the workspace lays down. Each chunk also installs a set of **skills** (real playbook files the agent discovers and invokes through Claude Code's own Skill tool) and a project memory, a `CLAUDE.md`, that points the agent at them. None of this is a bespoke protocol the agent had to be taught: it is a stock Claude Code project (settings, hooks, skills, memory), assembled fresh every chunk, and the agent works inside it exactly as it would in any local checkout. Temporal does not replace that ecosystem; it decides when the project runs, where, and which skills come with it.

When a chunk fails, the function does not decide what to do about it. It raises a typed error and lets the workflow's retry policy handle the backoff, and that policy is declared at the call site as configuration, not written as a loop. It is the same backoff that turned an overload window into added latency earlier, tuned for

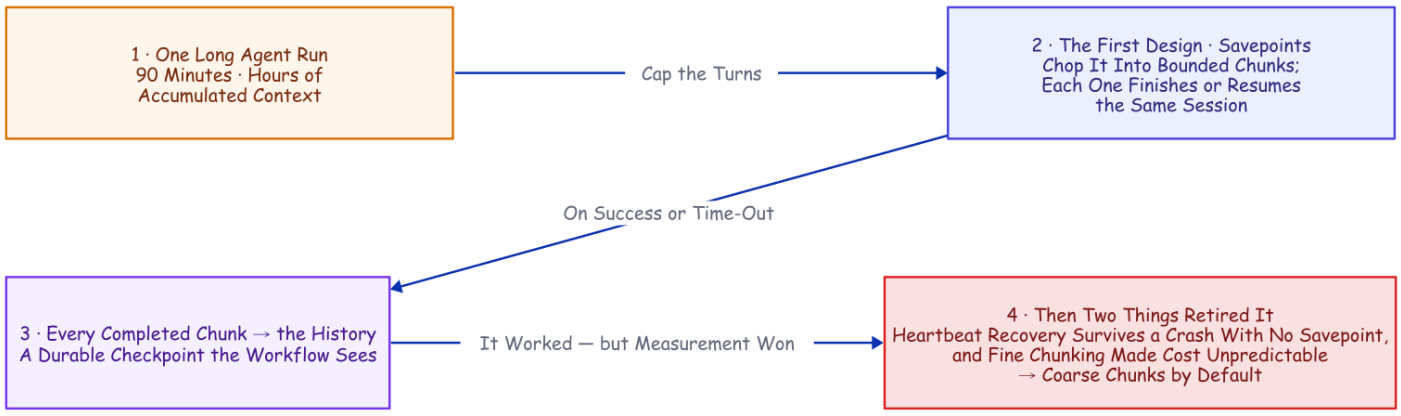
the API failure a headless run actually meets rather than the rare crash. The plain GitHub steps around it get a plainer policy; a refused merge is not an overload.[9]

The taxonomy under that is three-way, and it is very nearly the whole of the control logic. An API error or a mid-run failure is raised as *retryable*: the transcript survived, so the retry resumes the session instead of restarting the task. Running out of turns is not an error at all: the function returns normally and the workflow schedules the next chunk. Anything else terminal is raised as *non-retryable* and stops the job. Continue, resume, or stop: three outcomes a database could switch on, which was the entire point of sealing the agent away from the decision.

## The Detour We Measured Our Way Out Of

There is an older answer buried in this codebase, and it is worth showing precisely because it *worked*, until measurement talked us out of it.

An activity's result is all-or-nothing: nothing lands in the history until the attempt finishes. Wrap a ninety-minute agent run in one activity and a crash at minute eighty-nine erases everything the workflow could see. So the first design chopped the run into **savepoints** on that same turn cap. A capped run either finishes or runs out of turns mid-task, and in that second case the transcript survives, resumable by ID. Chain those together and you get a chunked session: each activity runs one bounded piece, and every completed piece writes progress, cost, and the session ID into the history. It worked: we killed a worker in the middle of the second of four chunks on a real to-do-app build, and it recovered on a restarted worker and finished all ten tests, one session end to end.



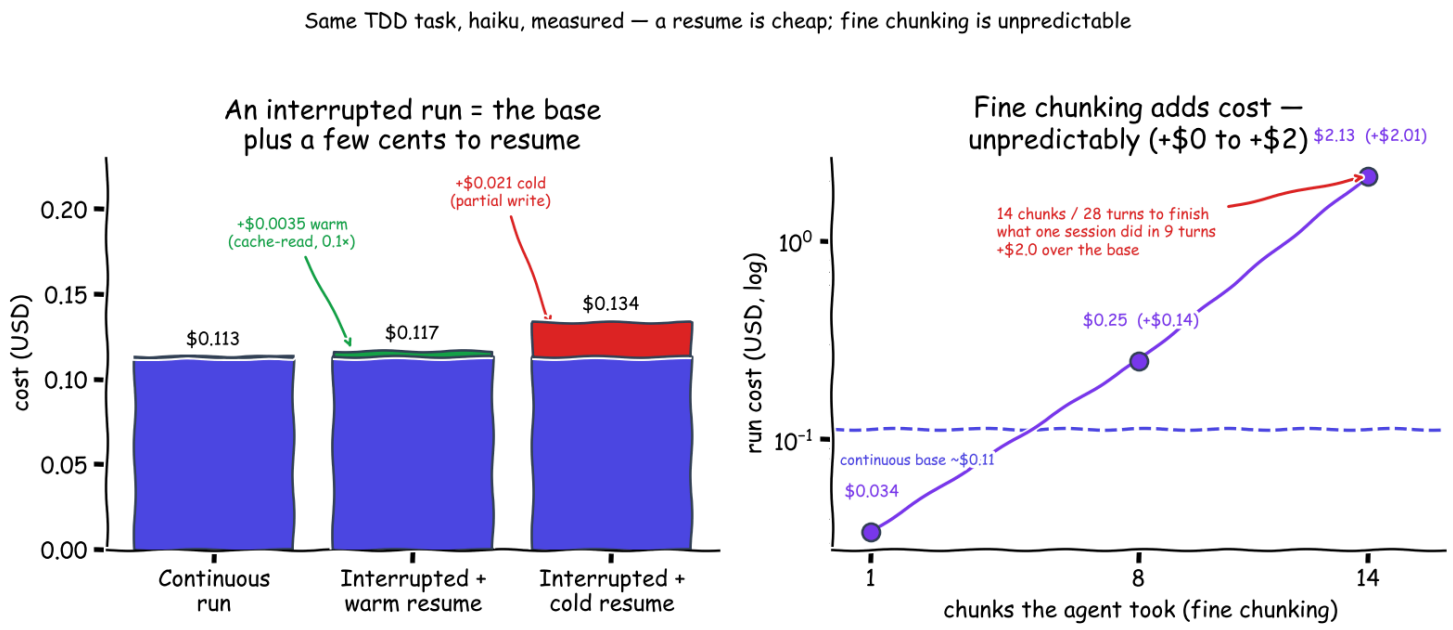

*The Savepoint Detour. The design was sound and the recovery was real. Two later facts retired it: heartbeat recovery survives a crash with no completed chunk at all, and, as the numbers show next, cutting the run into many small pieces makes the bill unpredictable.*

But every completed-chunk boundary is another resume: an agent invocation that re-establishes the growing conversation before it does any new work. Once heartbeat recovery existed, those boundaries were buying a

durability we already had. And they were quietly doing something worse to the cost, which is the part we did not see coming.

## What Durability Actually Costs

So we measured all of it directly: the same fixed test-driven task, on a small fast model, everything **observed rather than modeled**, run strictly one at a time. Three numbers matter: what a **continuous** run costs, what a **resume** adds when a crash interrupts one, and what **fine chunking** costs when you slice the run into many boundaries.




*What Durability Costs (measured, small model). Everything sits on top of the roughly eleven-cent continuous base. Left: after a crash, resuming only re-reads the accumulated context at the cache rate: the session's own cost, not a durability charge. Right: fine chunking adds anywhere from near-zero to two dollars, unpredictably, because a tight turn cap can send the agent to fourteen chunks to finish what one session did in nine turns.*

**A continuous run costs about eleven cents**: three runs came in at six, thirteen, and fifteen cents, with the agent taking anywhere from three to nine turns for the identical task. **Durability adds no tokens; a resume only re-reads context.** After a crash, resuming re-reads the accumulated conversation at the cheap cache-read rate: about a third of a cent warm, roughly 3% of the base run. Even a *cold* first touch after leaving the session idle for sixty-five minutes paid only a partial cache-write, about two cents, and then re-warmed. The API's extended-lifetime prompt caching keeps resumes cheap far past the usual five-minute window. That third-of-a-cent figure *is* the heartbeat-recovery number: a resume, not a restart from zero. (The live worker-kill from earlier, a separate and smaller task, landed at four cents total.)[10]

**Fine chunking is where the bill gets dangerous.** The same task, sliced into back-to-back two-turn chunks, cost $0.034, $0.25, and $2.13 across runs that took one, eight, and fourteen chunks: identical task, identical code, a sixty-three-fold spread, with the worst run costing nineteen times the continuous base. And

the culprit is not the mechanism we expected. The per-boundary cache re-read is *not* it; it stays a cheap fraction of a cent. The culprit is **behavioral**: a tight turn cap can send the agent to fourteen chunks and twenty-eight turns to finish what a continuous run did in nine. More turns, more output, and a prefix re-read at every seam. Coarse chunks simply do not hand it that much room to wander.

So the rule is **large chunks by default**: fine chunking is the only lever that meaningfully moves the total, and it moves it unpredictably. The baseline worth remembering is the bare loop with no durability at all: it pays the same eleven cents while nothing breaks, but a crash costs a full re-run, because the session ID died with the process. The durable coarse run pays the same bill and recovers for a fraction of a cent; a live issue-to-pull-request job confirmed it again, coming in at a clean eleven cents. The durability itself is effectively free. And notice what every number in this section has in common, because the pattern recurs across everything we measured: **the mechanics cost cents; the behavior costs dollars.** Resuming, recovering, chunking at a boundary are all pennies. The two-dollar surprise was something a model *chose to do with its turns*. Small chunks earn their keep for a different job entirely, which is next.

## Query, Steer, Cancel: What the Boundaries Are Still For

Completed-chunk boundaries did not stop being useful. They stopped being the durability mechanism, which freed them to be the *visibility and steering* cadence: the moments where the running job can answer to verbs a bare process cannot.

**Query it.** A read-only question, answered from the workflow's own state rather than from logs. Mid-run, a status query came back with the chunks completed so far, the running cost, and the live session ID. **Steer it.** A signal injected *"also add a stats subcommand, same test-driven treatment"* into a running to-do-app job; the workflow folded the instruction into the next chunk's prompt, and if a steer arrives during the very last chunk it triggers one more, so late guidance is never silently dropped. **Cancel it.** Delivered on the next heartbeat; the activity tells the running agent to stop, so it exits cleanly instead of burning tokens as an orphan, and the job lands in a canceled state you can see.

So the turn cap has one clear rule: it sets how often the job reports in and can be redirected. Make it small only when a human needs to watch or steer the run in near-real time. Everything else in the left column of the table below is something you would otherwise have built by hand:

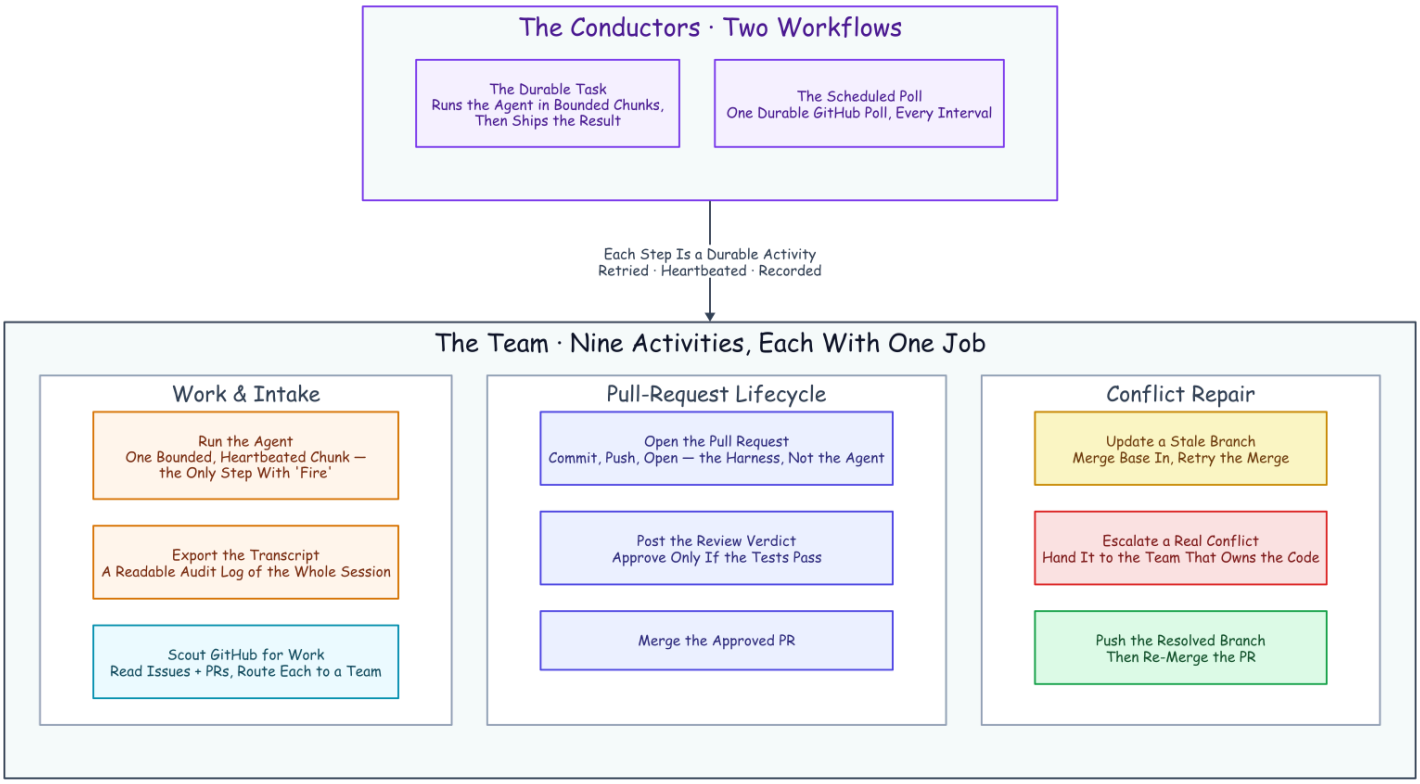
| What you would build by hand | The primitive that already exists |
|---|---|
| A session-ID file plus a lockfile | Job-ID uniqueness: one job *is* the session's single writer |
| A restart script plus a liveness probe | A heartbeat timeout plus a retry policy, declared not coded |
| A task-to-session map in memory and on disk | The session ID in heartbeat details mid-run, and in workflow state after a chunk |
| A hand-rolled taxonomy of which errors to retry | Typed retryable vs. non-retryable failures, with backoff |

| What you would build by hand | The primitive that already exists |
|---|---|
| A status endpoint scraped from logs | A progress query plus the event-history UI |
| Steering a running task (never actually built) | A signal, folded into the next chunk's prompt |

## A Team of Activities

Everything so far makes *one* agent durable. The reason to bother is that the same machinery, pointed sideways, makes a *team* of them, because a durable job, unlike a bare process, can be given an owner, an address, and rules. And the durability does not thin out as the team grows: every lane's agent runs inside the same crash recovery, the same resume-from-heartbeat, the same declared retries, so a nine-member team is nine durable jobs, not one durable conductor waving at fragile helpers.

Here is what we actually built. **Two workflows conduct the whole thing.** One is the durable task you have already met: it runs the agent in bounded chunks and then ships the result. The other is a scheduled poll that looks at a repository for new work, on a timer, as a durable job in its own right. And beneath those two conductors sits a **team of nine activities, each with exactly one job.** None of them is clever. Each is a sealed box that does one real-world thing and hands back typed data, and that narrowness is the point: it is what lets the retries, the heartbeats, and the audit trail apply to every one of them for free.




*The Team of Activities. Two workflows conduct nine one-job specialists. Read the columns as sub-teams: doing the work and finding it; the pull-request lifecycle; and repairing a conflict. Every box is a durable activity, so every box is retried, heartbeated, and recorded without anyone writing that code.*

Name the team by what each member does. One activity **runs the agent**: a single bounded, heartbeated chunk, the only member with any fire in its hands. One **exports the transcript** into a readable audit log once the work is done. One **scouts the repository** for new issues and pull requests and routes each to a team. Three handle the pull-request lifecycle: one **opens the pull request**, one **posts the review verdict** (approving only if the tests pass), one **merges** the approved change. And three handle the case where a merge fights back: one **updates a stale branch** by merging in the base and retrying, one **escalates a real conflict** to the team that owns the code, and one **pushes the resolved branch** so the merge can finally land. Nine specialists, two conductors, and a filed issue can travel the entire distance to a merged pull request while the humans sleep. And the shape should feel familiar: it is the discipline of a hardened CI/CD pipeline (gated merges, least-privilege credentials, an audit log on every step) applied to a team whose members are durable jobs.

The reason this is worth calling a *design* and not a script is that adding to the team is mechanical. Want a new capability (a security scan, a changelog write, a deploy)? You write one function that does the real-world thing and hands back typed data, you declare how it should retry, and you name the moment in the workflow where it runs. It inherits crash recovery, backoff, cancellation, and the audit trail from the harness, not from anything you wrote. The hard, distributed-systems parts are already solved; the only thing you add is the one honest step. That is the whole shape of "write the happy path and rent the rest."

## Trusted by the Queue, Not the Prompt

The team scales by giving each *kind* of work its own lane, and every "team" noun here is a concrete primitive underneath.

A **lane** is one Temporal namespace per team (a namespace is an isolated partition of the Temporal server; the teams in these runs were backend, frontend, and review — the repo now ships six lanes, discovered from its `teams/` folders) with its own work queue and its own bundle of skills bound into the workspace before each chunk runs. Workers listen only to the lane they own. And this is the part worth pausing on: a backend worker is trusted to do backend work not because its prompt says *act like a backend engineer*, but because it listens on the backend queue, runs under the backend skill bundle, emits backend audit artifacts, and answers to the same retry, cancel, query, and cost machinery as every other lane. **It is trusted by the queue it polls, not by the prompt it was handed.** The lanes are created from a script at startup, so the whole team topology lives in version control rather than being clicked into existence.

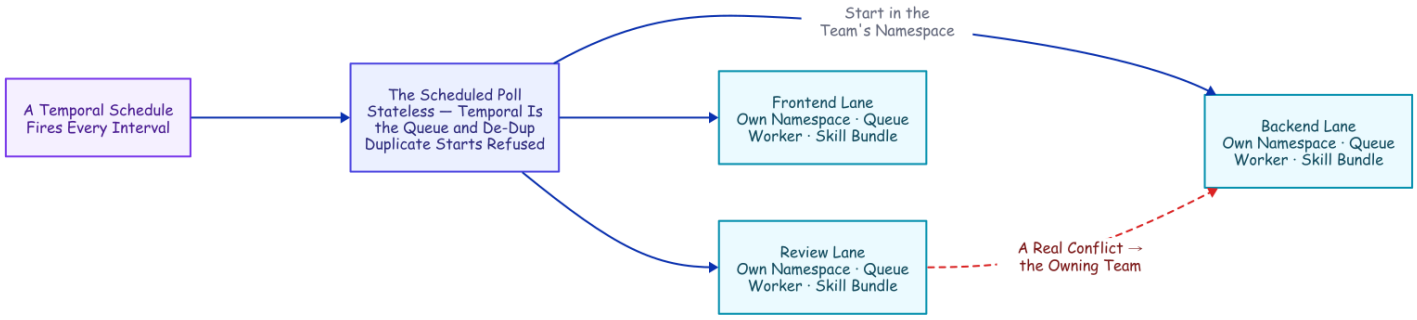
That skill bundle is not prompt flavor; it is a contract the lane owns. Each team's folder carries its mandate and its own copies of the disciplines it runs — test-first, review-your-own-diff, the required report format — tuned to the lane rather than cloned across it. The workspace binds to that folder live (the mandate is imported, the skills are linked), so the owning team's edits reach the very next chunk; only the policy files are stamped per chunk as an immutable snapshot.

The load-bearing rule lives in those bundles: a change in one layer has to prove it did not break the others.

- A backend change is obliged to run the frontend and end-to-end suites too, not only its own.
- The review lane will not approve a pull request until that cross-layer evidence comes back green.

That is where *trusted by the queue* earns the phrase: the queue decides which bundle a worker runs, and the bundle decides what the worker is on the hook to verify. It is also what makes each activity a genuine team member rather than a costume: the lane's skills are the playbook it actually follows for the work, while an identical org-wide deny floor governs every lane — and on top of that floor each team commits its own watched rules and its own finish gate, so a backend agent and a reviewer differ by discipline and by what their own team chooses to enforce, never by whether they are governed at all.

The formula for a team member, then, is short. **A mandate**: the one job it owns. **Skills**: the playbooks for how it operates. **Hooks**: the rules enforced on every tool call. **And a settings file stamped from human-committed harness code.** All four ride in version control; none of them lives in the model.



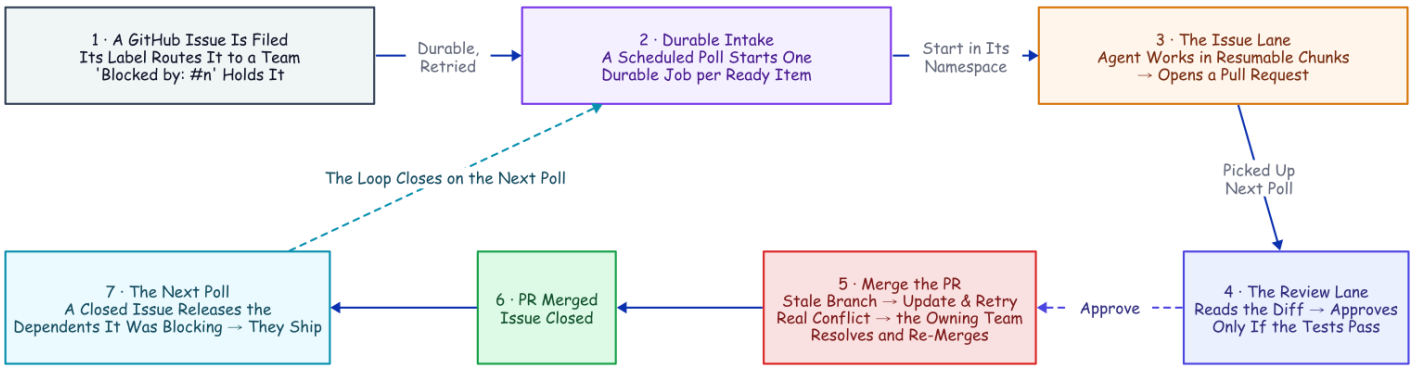

*Trusted by the Queue, Not the Prompt. Each team is a namespace with its own queue, worker, and skill bundle. The scheduled poll and the cross-lane escalation both reach another namespace the same small way: a client call made from inside an activity.*

The rest of the team's behavior falls out of a few more primitives. **Backpressure** is the queue itself plus a cap on how many chunks a single worker will run at once; you scale a hot lane by adding workers to it. **One owner per piece of work** comes from deriving each job's ID deterministically from the issue and starting it with an "allow duplicates only after failure" policy: a second attempt at a running or completed job is rejected by the server, while a failed job may retry on a later sweep. That single fact is *why* the poller can be completely stateless: re-seeing the same issue is a no-op the server refuses, so the poll keeps no memory of its own — and a transient failure heals itself instead of deadlocking the issue forever, a lesson a later fleet run taught the hard way (`deploy/fleet-run-results.md`). **Intake** is a Temporal schedule firing the poll workflow, so even "check the repo for work" is a durable, retried, observable job instead of a cron line that can silently die. And **sequencing** is a plain "blocked by #n" line the poller honors, holding a dependent issue until its prerequisite closes.

One structural catch shapes the whole picture. A workflow can only start child workflows inside its own namespace. Both the scheduled intake and the escalation below have to cross a namespace, and they do it the same modest way: from *inside an activity*, which is allowed to do arbitrary I/O, open a client to the target namespace and start the job there. The deterministic ID and the duplicate policy keep it idempotent even across that boundary.

## From Filed Issue to Merged PR

Put the conductors and the team together and one issue travels a full loop, every box along the way a durable step in the history.




*From Filed Issue to Merged PR. Every box is a real activity from the team. The conflict detour and the unblock loop are not special cases; they are the same durable machinery running one more time.*

**Intake.** A schedule fires the poll on an interval. The poll reads the repository (open issues, open pull requests, and which issues have already closed), routes each item to a team by its `team/…` label (or the catch-all lane when it carries none), and starts one durable job per ready item with a deterministic ID. An issue marked *blocked by* another (a line in its body or a `blocked-by` label) is held until that other issue closes.

**Build.** The issue's job runs the agent in bounded, resumable chunks, heartbeating its session ID the whole time. When the agent reports success, the transcript is exported to a readable audit log and a pull request is opened from the work branch.

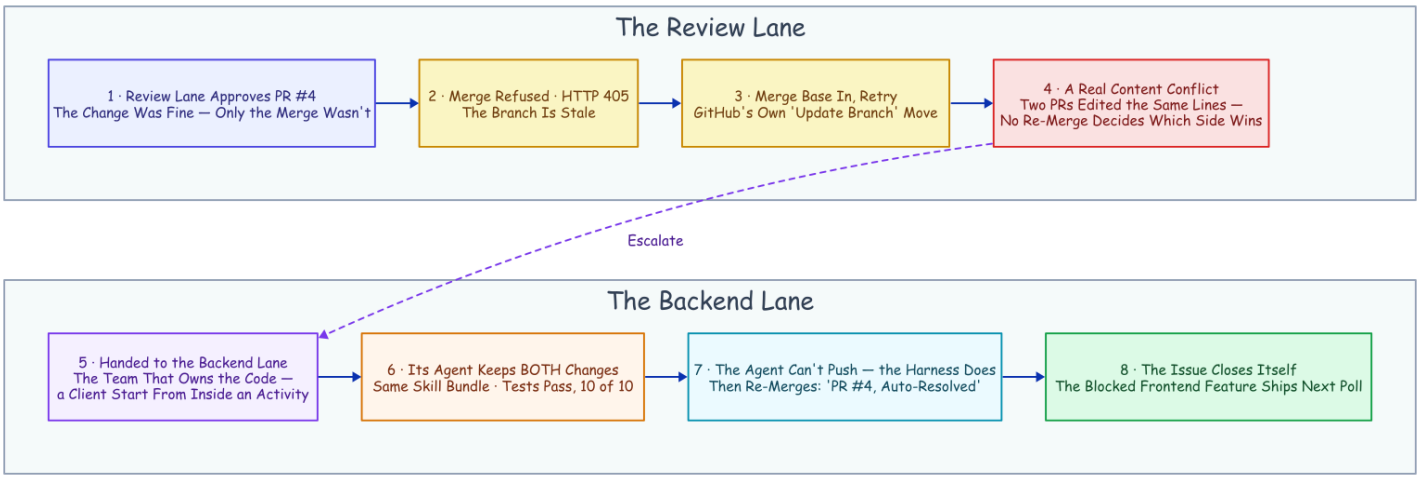
**Review and merge.** That open pull request is picked up on the next poll and routed to the review lane as its own job. The agent reads the diff and runs the suite, and its verdict is a single field in the required report: a pass/fail boolean that *is* the merge switch. True posts an approval and merges the change; false requests changes. A merge against a branch that has fallen behind triggers an automatic branch update (the same "update this branch" move you would click in the GitHub UI) and a retry. And the false case does not have to be a dead end: with the optional fix loop on, a red verdict (or a human's Request Changes) hands the PR back to the owning lane to fix, re-validate, and re-review, re-asking a human when the companion's merge gate is also on, bounded to a few rounds; the mechanics are in the companion piece.

**Escalate, resolve, unblock.** A retry that *still* conflicts is a genuine content collision, and it gets handed to the team that owns the code: a resolve job in that team's own namespace, where an agent merges in the base, fixes the conflict by hand, runs the tests, and then the harness pushes the fixed branch and lands the merge. When that merge closes the issue, the next poll releases whatever was blocked on it, and the chain runs again for the dependent. No human decision anywhere in it.

## The Conflict That Resolved Itself

Every piece of that machinery was about to be needed at once, so we built a real target for it: a full-stack "snippets" web app, backend and frontend and browser tests, wired to a live deployment with a namespace per team and the poller on a schedule.

Two backend issues landed in the same poll window and collided by construction. Both edited the same region of the same backend file: one added search, the other added an endpoint for deleting a snippet. Each got its own branch, its own job, its own pull request. Search merged first, and the platform immediately marked the delete request as conflicting. This is the exact spot where autonomy usually ends quietly: a "needs a manual rebase" comment, and a human inheriting the mess in the morning.




*The Conflict That Resolved Itself. Read it top row then bottom row. Detection lives in the review lane; the fix lives in the lane that owns the code. The escalation is the same cross-namespace client call the scheduled poll uses.*

What happened instead, every step a durable activity in the history: the review lane approved the pull request on its merits, because the change itself was fine and only the merge was not. The merge attempt was refused. The workflow's first response was the boring, correct one (merge the base branch in and retry), and that surfaced a **real content conflict**: two pull requests had edited the same lines, and no amount of re-merging the base decides which side should win. That is a judgment call, and judgment is what the agent is for.

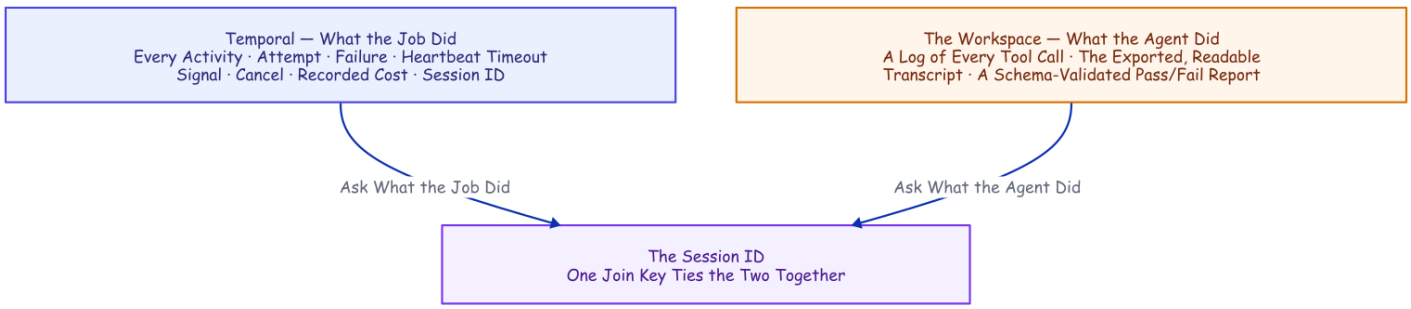
So the review workflow escalated, not to a human but to the team that owns the code. It read the issue number out of the branch name, looked up that issue's team, left a comment naming the resolve job, and started that job **in the backend namespace, on the backend queue**, the same client-from-inside-an-activity call the scheduled poll uses. The backend agent that picked it up was nothing special: same skill bundle, same policy, same durable harness, only the prompt was different. It merged the current base, kept both the base's changes and the branch's intent, ran the tests green on the merged tree (ten of ten), and, its own push denied by policy, left the harness to push the branch and re-merge it. *"Merge PR #4 (conflict auto-resolved)."* The issue closed itself a moment later.

Two properties keep this from being a party trick. The loop cannot run away: the resolve job's ID is deterministic (keyed by the PR *and* its current head commit, so a resolved push gets a fresh review and cascading conflicts converge), and duplicates of a live or finished job are refused by the server — a second escalation ten seconds later is a no-op. And nothing traded durability for the cleverness: the escalation is a workflow starting a workflow, subject to the same crash recovery and the same hard limits as everything else.

Then the part that turns an incident into a system. A frontend issue had been sitting in the queue the whole time, marked *blocked by* the very issue that just closed. On the next poll the hold released itself; the frontend agent built a delete button against the exact endpoint the resolution had just landed, added a browser test, and opened a pull request the review lane merged. From collision to shipped downstream feature, the chain ran without a human decision in it, one mechanical asterisk included.[11]

## Every Run Leaves Evidence

Because the agent runs under a policy the workspace owns, every run leaves *two* planes of evidence, joined on one key: the session ID. Temporal owns the record of the **job**: every activity, attempt, failure, heartbeat timeout, signal, cancel, recorded cost, and session ID, queryable long after the run is over. The workspace owns the record of the **hands**: a hook appends every tool call to a log as it happens, a closing activity exports the whole conversation to readable Markdown filed under the session ID, and the required pass/fail report is stored as typed data the workflow reads as a real value rather than a vibe.



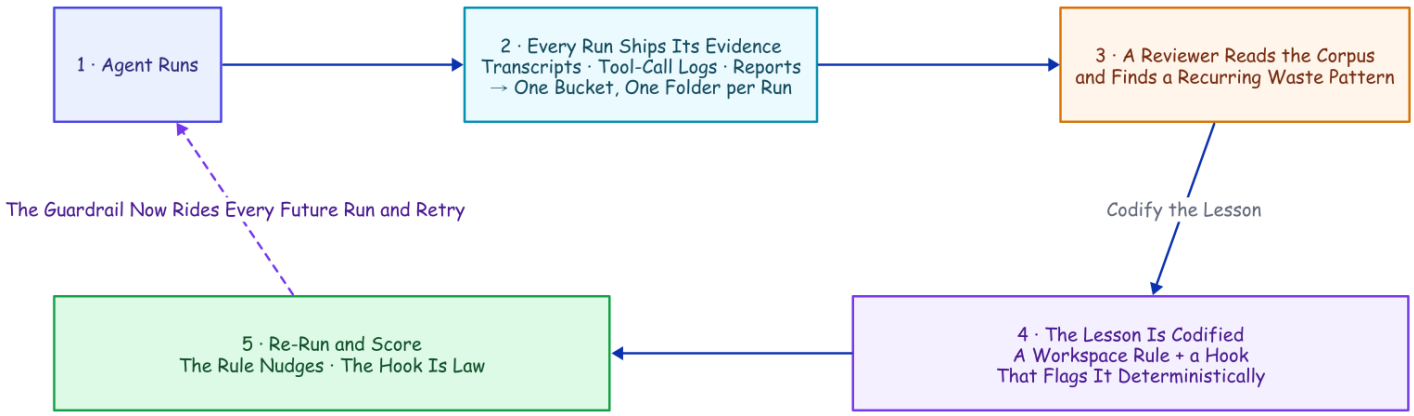

*Every Run Leaves Evidence. Ask what the job did and you read Temporal's history; ask what the agent did and you read the workspace's package. One session ID ties them together, because the job is a durable object and not a vanished process.*

The policy rides *inside* the workspace, not on the machine, and its author is a human: the version-controlled configuration the workspace rewrites before every chunk is what keeps the deny rules, the hooks, and the skills identical across every worker, retry, and resume. One hygiene detail is worth stealing: the activity clones the repository with a token and then scrubs that token out of the remote, so the agent cannot recover the clone credential from the repo's own config. A companion detail runs the other way: before any push, the harness excludes its own scratch (the configuration folder, the memory file, the report, every audit log) from the commit, so the control plane's policy and its paper trail never ride inside the product it ships. When work needs pushing, the *harness* pushes, with its own credentials, in a separate step. The agent writes the code; the control plane is the only thing that touches the outside world.

## The Corpus Improves the System

Every run exports its conversation (transcript, tool-call log, report), and once you collect those exports, you have a corpus of how your agents actually behave. Shipped to object storage, that corpus outlives any worker;

the six runs behind the cost numbers above went to a real bucket, one folder of evidence per run.[1][2] A corpus is a feedback loop waiting to be closed, so we closed it, and the loop has a shape worth naming: runs leave evidence, a review finds a mistake, the mistake becomes code, a re-run scores the fix, and the fix rides every future run. Nothing the system gets wrong is allowed to stay a lesson.




*The Corpus Improves the System. Runs become a corpus in a bucket; a reviewer finds a recurring waste; the lesson becomes a guardrail that rides every future run; a re-run scores it.*

A reviewer read the tool-call logs across all six runs and found one consistent waste: the agent kept re-checking work it had already done, listing a directory to confirm a file it had just written (the write already proved the file exists) and re-running tests that were already green. The leanest runs finished in eight or nine tool calls; the worst burned twenty-four, dominated by this re-checking. We codified the lesson two ways, both into the workspace bootstrap so they ride every future run and retry: a plain rule in the project's instructions ("trust your tools; don't list a directory to confirm your own work; run the suite once"), and a hook that deterministically flags any orientation-only directory listing.

Then we re-ran and scored the before and after, and the result is better than a clean win. The soft rule produced only a **modest ~20% drop** in the targeted pattern (from 2.5 to 2.0 instances per run on average, a half-instance change that sits within run-to-run noise), and one run ignored it entirely; the total tool count did not fall at all (it in fact rose, from 15.5 to 18.5 calls per run), swamped by ordinary nondeterminism. The hook flagged **every remaining instance, twelve out of twelve**, regardless of whether the agent complied. The measurement makes the distinction precisely: the prompt rule is probabilistic, while the hook is deterministic. It is the same asymmetry the deny rules bought at the start of this article, now with numbers attached, and it compresses to the one sentence we would keep if we could keep only one: **a line in a prompt is a suggestion; a hook is a law.** The useful result is not that the agent improved on average, but that the mistake is now caught on every run.

## What This Doesn't Solve

Six real gaps, told plainly, because the honest ones are the load-bearing ones.

- **Worker affinity.** The transcript and the workspace are local files, and resume only works where they are visible. The engine can recover the *workflow* anywhere, but a retried chunk has to land where the *session* lives. The fixes escalate: one worker per lane; a per-worker queue chosen on the first chunk; shared storage for the transcript directory and the workspace.
- **The filesystem is not checkpointed, and steps run at-least-once.** The orchestration state is durable and the transcript survives, but the working directory is neither versioned nor rolled back, so a retried chunk resumes against a directory the dead attempt half-mutated. It converged in every test here, but *converges in practice* is not *idempotent by construction*. Any outward action (a push, a comment, a merge) wants an idempotency key, and the structural fix is a fresh git worktree per chunk: commit on success, reset on retry.
- **A hard kill can orphan the agent.** Kill the worker and the agent's child process may finish its chunk anyway, racing the retry: the exact effect that masked our first recovery attempts. Production workers belong in a process group or a container that takes their children down with them.
- **Workflow code is a compatibility surface.** Deterministic replay cuts both ways: reorder the steps while runs are in flight and replaying an old history against new code can wedge the run. Evolving an orchestration in production is a versioning discipline, not just editing a function.
- **A permissions landmine.** The agent's most permissive mode combined with resume in headless mode was broken upstream and has since been marked fixed;[13] re-check it against your version. This project sidesteps it entirely with a mode that auto-accepts file edits but still enforces an explicit allow-list for everything else.
- **The merge gate is an agent's verdict.** Approve-and-merge here is gated on a single schema-validated boolean: the review agent's tests-pass verdict. For this demo repository, that is the point; for production, it is a placeholder, not a posture. A real deployment should insert its human gate exactly there: a protected branch that requires human review, or a durable approval step the workflow waits in. That durable approval step is actually built and verified against a live server, modeling the person on the same query, update, and timer primitives as everything else, deny-safe when nobody answers; the companion piece *The Human Is a Durable Object* is the walkthrough and the live run.

## When Bare Claude Code Is Enough

Durability is insurance, and you do not insure a cheap errand. The alternative to this harness is not a lousy while-loop; it is Claude Code's own considerable durability, and sometimes that is the right answer.

- **The task is cheap to re-run from scratch.** A one-chunk, two-minute job needs a retry button, not an engine. The plain headless command plus your CI's retry is the correct architecture.
- **One developer, one machine, interactive.** The transcript plus a resume by hand *is* the durability layer. A workflow engine on a laptop is ceremony.
- **You would rather buy the orchestration than own it.** If owning the loop is not strategic for you, a managed agent runner is a legitimate answer to the same problem.

The threshold is crisp. The moment a task **outlives the process that started it** (an overnight run, a queue of issues, anything a deploy or a crash can interrupt, anything whose re-run from zero is a real bill), something has to remember the job. Then you have exactly three options: lose the state and eat the tokens; hand-build the state machine and debug it by incident; or use an engine whose entire product is that state machine.

## Defaults Worth Stealing

If you build any version of this, on any stack, these are the settled defaults this project would hand you. Each one was paid for above.

- **Big chunks by default.** A tight turn cap is a slot machine: the same task ran $0.034 to $2.13 on identical code. Chunk for visibility and steering, never for durability.
- **Heartbeat a resume handle, not just liveness.** The last pulse is a free checkpoint; a retry that can read it never starts from zero.
- **Kill, and deploy, by process group.** An orphaned agent child finishes the work anyway and lies to you about whether your recovery works.
- **Deny `git push` to every agent.** The harness, with its own credentials, in its own recorded step, is the only hand that touches the outside world.
- **Route by queue, not by prompt.** A lane's queue and skill bundle are an identity a worker cannot drift out of; a persona in a prompt is not.
- **Deterministic job IDs, duplicates allowed only after failure.** Running and completed jobs are refused, so the poller needs no memory and the escalation loop cannot run away; failed jobs may retry on a later sweep, so one transient error cannot deadlock an issue forever. (A live fleet run found the stricter refuse-everything version of this rule doing exactly that; `deploy/fleet-run-results.md` has the evidence.)
- **Write the rule as a hook, not a prompt.** The written rule nudged the waste pattern down ~20% and was ignored once; the hook flagged twelve of twelve.

## The Harness Was the Hard Part All Along

One practitioner teardown of Claude Code's own codebase estimates that roughly **1.6%** of it is the AI decision logic. The other **98.4%** is operational harness: permissions, state, recovery, tools.[14] Set that number next to the hand-build list from the start of this article and the overlap is hard to miss. The harness every agent team is rebuilding is, almost line for line, what durable-execution engines have provided for a decade.

The two sides are already circling each other. Temporal's own AI cookbook wraps model *API calls* in activities; a production write-up wraps the agent framework the same stateless way; another project built durable checkpointing around whole Claude Code runs on its own runtime.[15][16][17] Each holds one half of the seam. In the sources we checked, none composes the two the way this project does: headless Claude Code

*sessions*, with the session ID carried in heartbeat details mid-chunk and in workflow state after a completed one. That is a claim with a short shelf life, so re-check it before you build.

The practical takeaway is narrow. Once a task outlives the process that starts it, the durable-execution engine is the part worth adopting, and the agent belongs inside it rather than inside a hand-built loop.

But the composition is the natural one, and the seam really is small. Claude Code brings the durable conversation. Temporal brings the durable job. The agent could always remember the conversation; everything in this article is what became possible the moment something remembered the job.

---

*Disclaimer: the views and opinions expressed in this article are my own and do not necessarily reflect those of my employer. This is a personal project, not affiliated with or endorsed by Anthropic or Temporal.*

## Sources

- **Claude Code headless mode, sessions, CI action, permissions, sandboxing**: code.claude.com/docs (headless, sessions, github-actions, permissions, hooks, sandboxing, agent-sdk). *Vendor/canonical.* Checked 2026-07-15.
- **Temporal durable execution, activities, signals/queries, heartbeats, heartbeat throttling, workflow versioning**: docs.temporal.io and python.temporal.io. *Vendor/canonical.*
- **Measured runs.** The four-cost experiment (continuous ~$0.11; warm resume $0.0035; cold resume $0.021; fine-chunked $0.034–$2.13), the failure→heartbeat→resume recovery, and the corpus/learn-loop before/after are reproducible from the public repo, github.com/thumarrushik/temporal-claude-demo: `deploy/full-experiment-results.md`, `deploy/heartbeat-recovery-results.md`, `deploy/learn-loop-results.md`, with runners and analyzers under `deploy/`. Agent model `haiku`, 2026-07-15.
- **Most-permissive mode + resume bug**: github.com/anthropics/claude-code issue #36139. *Vendor issue tracker.* Re-check before reuse.
- **1.6% / 98.4% harness split**: *Dive into Claude Code* teardown, arXiv 2604.14228. *Practitioner/academic.*
- **Temporal AI cookbook (model API in activities); agent-framework × Temporal production guide; durable Claude Code checkpointing on another runtime**: docs.temporal.io/ai-cookbook; claudelab.net (2026-04-22); zenml.io/blog (2026-06-01). *Vendor + practitioner.*

---

1. Claude Code headless mode: `--output-format json` returns `session_id`, cost, and (with a JSON schema) structured output; `--resume <id>` continues a stored session; transcripts are stored under `~/.claude/projects/<cwd-slug>/<session-id>.jsonl`, the working-directory path with non-alphanumerics replaced. Checked 2026-07-15. *Vendor/canonical.* ↩
2. `anthropics/claude-code-action@v1` (GA) runs headless Claude Code on `pull_request`/cron and on `@claude` mentions, with automatic PR review; the docs note it is built on the Claude Agent SDK. Checked 2026-07-15. *Vendor/canonical.* ↩

3. Claude Code permissions are evaluated deny → ask → allow (a deny is unoverridable) and "enforced by Claude Code, not by the model"; `PostToolUse` hooks receive the tool event as JSON and can block a call; Bash sandboxing uses Seatbelt (macOS) / bubblewrap (Linux). Checked 2026-07-15. *Vendor/canonical.*↩

4. Claude Code sessions: resume lookup "is scoped to the current project directory and its git worktrees"; there is no documented cross-machine session transport or leasing: the gap an external orchestrator must fill. Checked 2026-07-15. *Vendor/canonical.*↩

5. Temporal docs: durable execution persists workflow progress as an event history; workflows must be deterministic so history replay reconstructs state; activities host side effects and are retried by policy; Temporal descends from Cadence, built at Uber. *Vendor/canonical.*↩

6. Temporal Python SDK: `activity.heartbeat(*details)` sends heartbeat details, and `activity.info().heartbeat_details` exposes the previous attempt's details to a retry. The worker throttles how often details are persisted (`max_heartbeat_throttle_interval` / `default_heartbeat_throttle_interval`). Checked 2026-07-15. *Vendor/canonical.*↩

7. `deploy/heartbeat-recovery.sh`, 2026-07-15: the worker's whole process tree was SIGKILLed mid-chunk before any completed result; attempt 2 recovered the heartbeat session ID (with the input session ID null) and relaunched with resume; it completed as one chunk, $0.0404, the same session. Requires a heartbeat throttle short enough to persist the ID, and a process-group kill so the agent child can't orphan-finish the work.↩

8. With a heartbeat timeout set, the effective throttle is `min(heartbeat_timeout × 0.8, max_heartbeat_throttle_interval)`; the worker sets both throttle knobs from an environment variable (default 15s) so the in-flight session ID is recorded promptly. See `src/worker.py`.↩

9. The workspace policy is human-committed in each team's folder — `teams/<team>/.claude/settings.json` (the deny rules, the two `PostToolUse` hooks, and a `Stop`-hook phase gate added after these runs) plus the lane's `rules.json` — and `src/activities.py` stamps it into the workspace each chunk, injecting absolute-path denies that protect the `teams/` source itself. The project-only config load (`setting_sources`), the accept-edits mode with an explicit tool allow-list, and the schema-required report live in `src/activities.py`. The per-call retry policy (5s, doubling to a 2-minute cap, 6 attempts), the two-minute heartbeat timeout, and the fifteen-minute chunk ceiling are set where the workflow invokes the chunk in `src/workflows.py`.↩

10. `deploy/full-experiment.py`, 2026-07-15, model `haiku`, run strictly sequentially; everything observed, not modeled. Continuous measured ×3 ($0.058/$0.132/$0.149); warm boundary via no-op resume ×5 ($0.0035, cache-read); cold boundary via no-op resume after a 65-min idle gap ($0.021, partial cache-write); fine-chunked totals ×3 ($0.034/$0.25/$2.13 at 1/8/14 chunks). Cost is the CLI's own `total_cost_usd`, identical on a laptop or a cloud VM. Full write-up: `deploy/full-experiment-results.md`.↩

11. The asterisk: that frontend pull request predated the escalation code, so its review was re-triggered against the new code, and the final polls were hand-triggered rather than waited out on the timer. The scheduled poll and the first-pass routing are proven across the other recorded runs; this run isolates the resolution chain.↩

12. `deploy/learn-loop-results.md`, 2026-07-15: the six cost-experiment runs' transcripts/tool-logs/reports were pushed to `gs://temporal-claude-corpus-213476/baseline/`; a reviewer identified redundant directory-listing re-orientation; the lesson was codified as a workspace instruction rule plus a `PostToolUse` flag hook (`src/activities.py`); a re-run scored the modest rule effect and the deterministic hook (12 flags).↩

13. anthropics/claude-code#36139: bypass-permissions mode misbehaves with `--resume` in print mode; reported and marked resolved (re-check against your installed version). This project uses an accept-edits mode plus an allow-list plus workspace deny rules instead. *Vendor issue tracker.*↩

14. *Dive into Claude Code* (arXiv 2604.14228): ~1.6% of the codebase is AI decision logic; ~98.4% operational harness. *Practitioner/academic.*↩

15. Temporal AI cookbook: the "Basic agentic loop with Claude" uses model Messages API calls as activities; no Claude Code CLI, no session resume. *Vendor/canonical.*↩

16. "Building fault-tolerant long-running AI workflows with Claude Agent SDK × Temporal.io," claudelab.net (2026-04-22): wraps API messages in activities; no headless sessions, no cross-activity resume. *Practitioner.*↩

17. "Don't make Claude do the same work twice," zenml.io (2026-06-01): durable checkpointing of Claude Agent SDK runs on ZenML's own runtime, not Temporal. *Practitioner.*↩